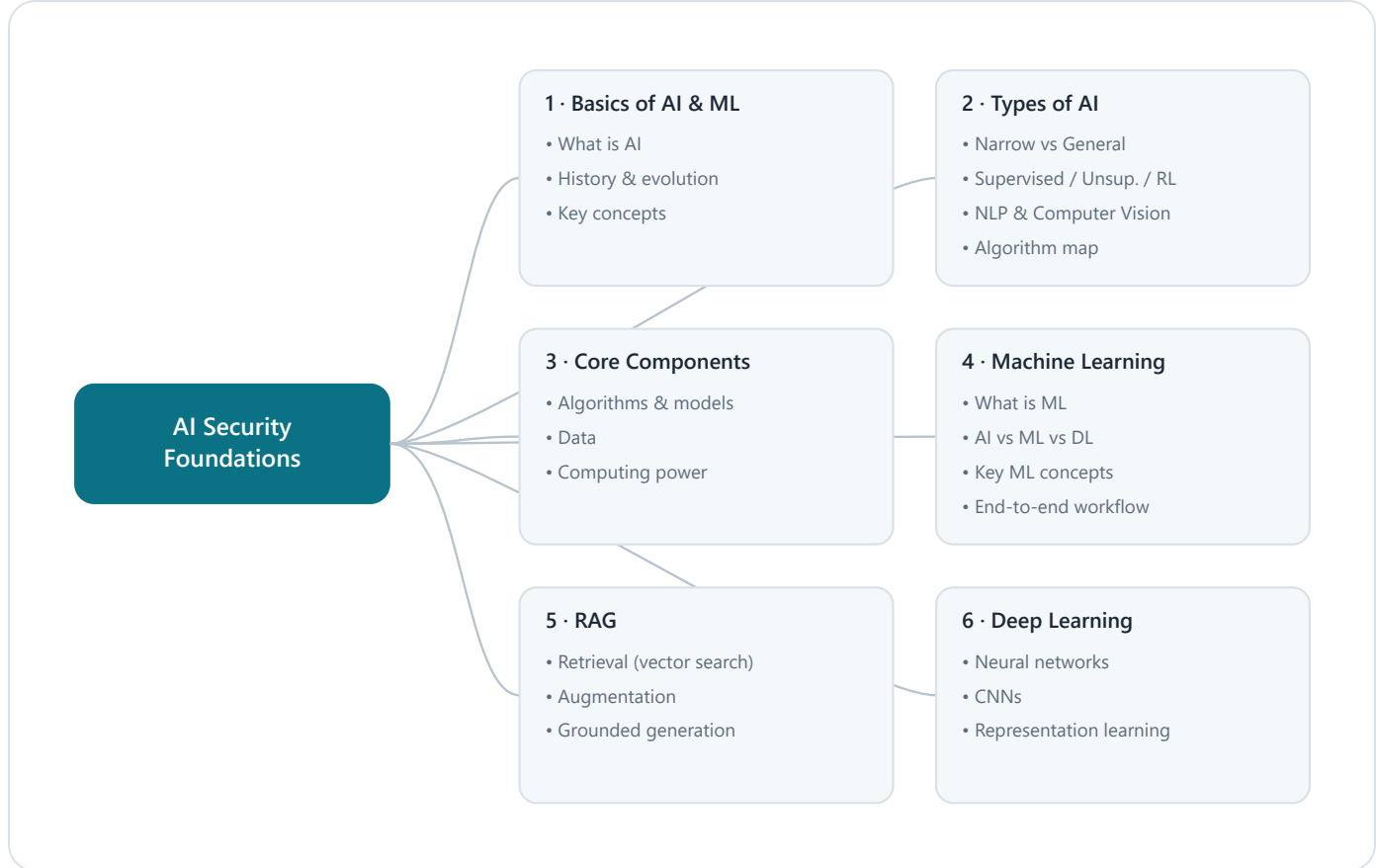


An Overview of AI Security — Foundations

The AI/ML fundamentals you need before AI security proper. Every concept is explained on its own terms, then re-examined through a **Security Lens** that connects it to threat modeling, detection, and defense. All diagrams are inline SVG or embedded images, so this single file renders anywhere — no plugins, no external assets.

Contents

- | | |
|---|---|
| 1 Basics of AI and ML | 5 Retrieval Augmented Generation |
| 2 Types of AI | 6 Basics of Deep Learning |
| 3 Core Components of AI Systems | A AI Security Attack Taxonomy |
| 4 Introduction to Machine Learning | B Governing Frameworks |
| | C Python Toolkit by Concept |



The six topic areas, each carrying its own security lens.

Section 1

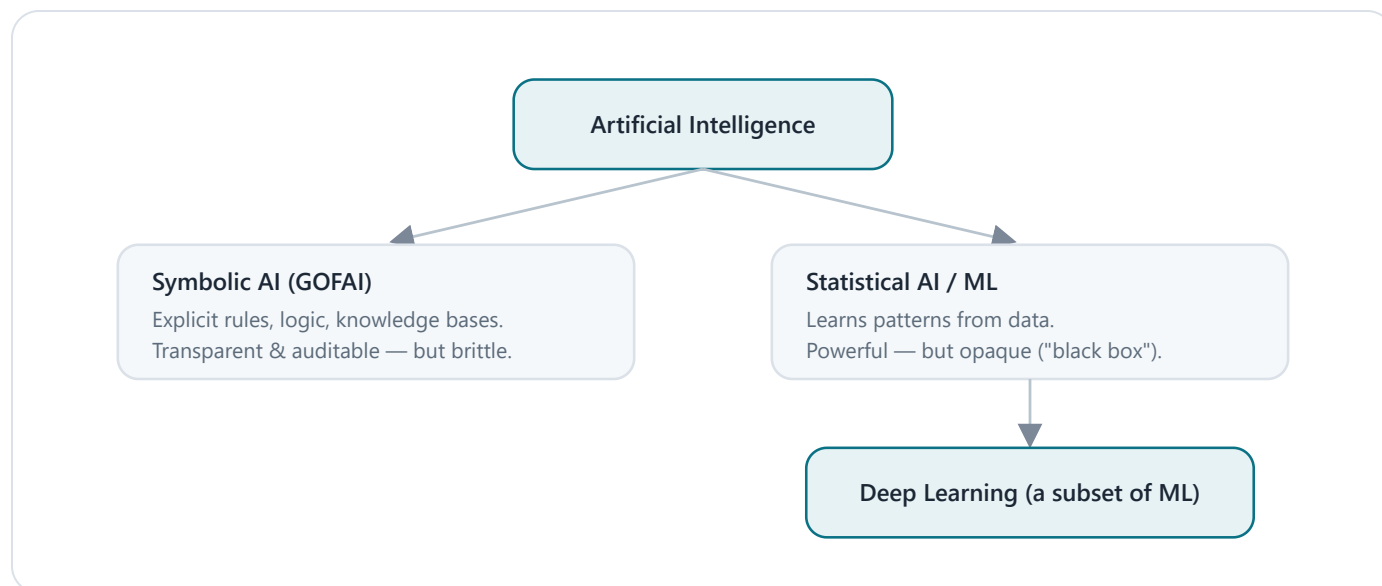
Basics of AI and ML

1.1 · What is AI?

Artificial Intelligence (AI) is the field of building systems that perform tasks normally requiring human intelligence — perceiving, reasoning, learning, planning, understanding language, and making decisions.

Most modern engineering uses the **rational agent** view: an agent perceives its environment through sensors and acts through actuators to maximize its expected performance. Two axes help frame "intelligence": *thinking vs. acting* (does it reason like a human, or just behave usefully?) and *human-like vs. rational* (does it imitate humans, or act optimally toward a goal?).

The two great families of AI



Symbolic systems encode logic you can read; learned systems hide their logic in numeric weights.

Security Lens — why the definition matters

A symbolic system's logic can be reviewed like source code. A learned model's "logic" lives in millions/billions of numeric weights that cannot be directly inspected. This opacity is the root cause of most AI-specific risk: you often can't tell *why* a model decided something, whether it memorized sensitive data, or whether an attacker nudged it. Your threat model shifts from "review the code" to "reason about data, training, and inference behavior."

1.2 · History and Evolution of AI



Amber dots mark the two AI winters. Each boom introduced new capability — and a new attack surface. The latest one: agents that take real actions.

The story to be able to tell: philosophical foundations (1943–56) named the field at Dartmouth; early optimism around the Perceptron cooled when its limits (the XOR problem) were exposed, triggering the **first winter**; rule-based **expert systems** over-promised into a **second winter**; the field then shifted from hand-coded rules to *learning from data*; **AlexNet (2012)** proved deep networks + big data + GPUs was the winning formula; the **Transformer (2017)** unlocked the LLM/generative wave; and since 2024 the frontier has moved to **reasoning models** and **agents** that call tools, browse, write code, and act in real systems.

Security Lens — the tech always outran its defenses

Expert systems introduced tamperable rule bases; statistical ML introduced data poisoning and adversarial inputs; LLMs/RAG introduced prompt injection and training-data extraction; agents turn a successful injection into *actions* — tool misuse, privilege abuse, data exfiltration. Security has trailed capability at every step — you're catching up on a decade of accumulated, under-defended surface.

1.3 · Key Concepts in AI

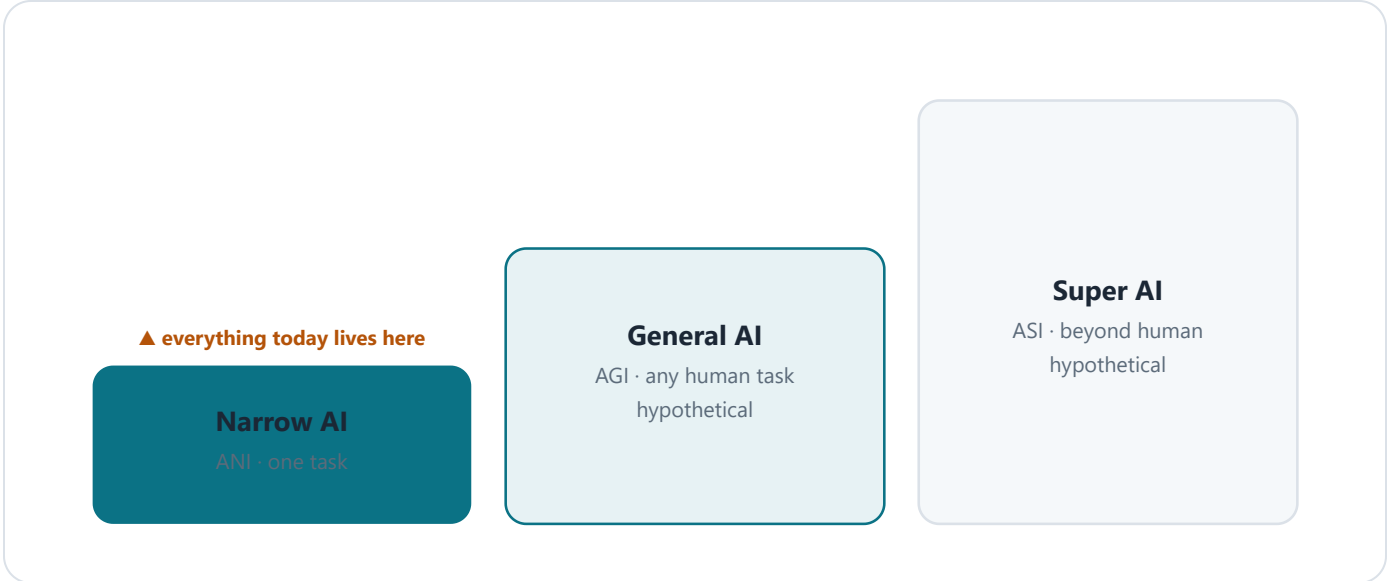
- **Agent & environment** — anything that perceives (sensors) and acts (actuators). Described with **PEAS**: Performance measure, Environment, Actuators, Sensors.
 - **Rationality** — acting to maximize *expected* performance given what the agent knows (not omniscience).
 - **Search & optimization** — exploring possible states/solutions efficiently, often with heuristics.
 - **Knowledge representation & reasoning** — how facts are stored and how new conclusions are inferred.
 - **Learning** — improving performance from data/experience (the ML core).
 - **Uncertainty** — real-world agents reason probabilistically (Bayesian methods).
-

Section 2

Types of AI

There are **two orthogonal** ways to classify AI. Don't confuse them: one is about *how general* the system is; the other is about *how it learns*.

2.1 · Narrow AI vs. General AI



Capability tiers. Today's frontier models are general-purpose across many tasks, but AGI is not established — whether and when it arrives is actively debated.

	Narrow AI (ANI)	General AI (AGI)	Super AI (ASI)
Scope	One specific task/domain	Any task a human can do	Surpasses humans broadly
Status	Everything widely deployed today	Not established; definitions and timelines contested	Hypothetical
Examples	Spam filters, image classifiers, AlphaGo; frontier LLMs and agents are the gray zone	—	—

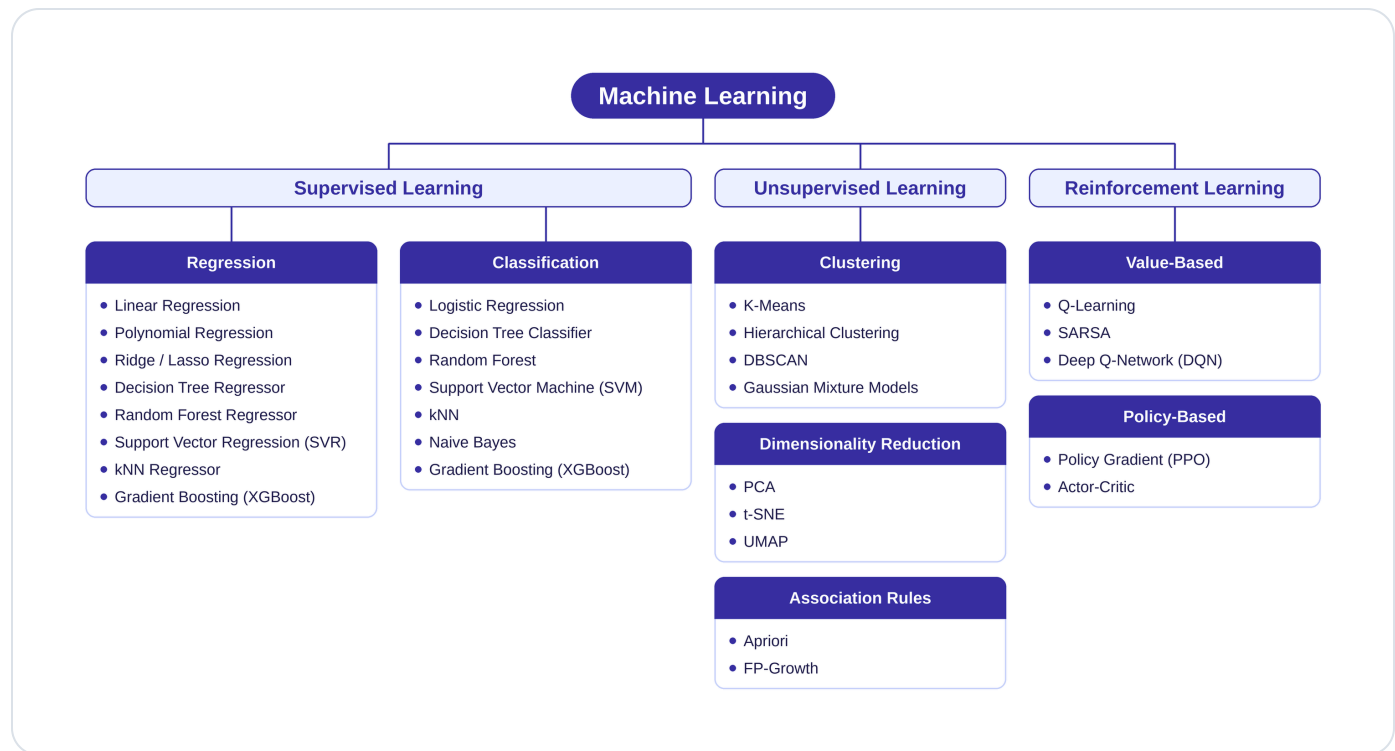
 Security Lens

Nearly all operational risk is *narrow-AI risk, here and now* — poisoning, evasion, prompt injection, data leakage. Keep threat modeling grounded in deployed systems; AGI-alignment debates are a different problem class than defending this quarter's production model.

Learning paradigms at a glance

The second way to classify AI is by *how it learns*. The map below shows the three paradigms covered in Sections 2.2–2.4 and the algorithm families you'll meet most

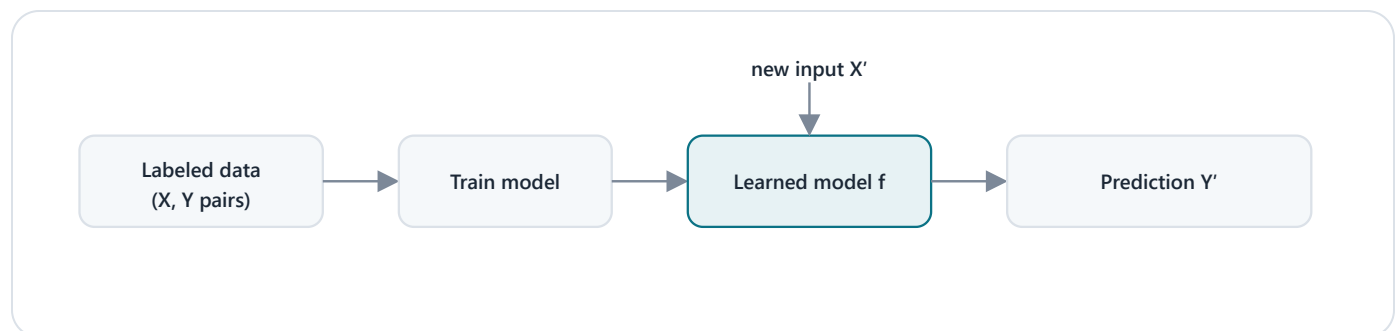
often under each. Supervised learning splits by output type (a number vs. a class); unsupervised learning splits by the structure it finds; reinforcement learning splits by whether the agent learns action values or the policy directly.



Common algorithms by paradigm. Several (decision trees, random forest, SVM, kNN, gradient boosting) come in both a regressor and a classifier form.

2.2 · Supervised Learning

Learn a mapping from inputs **X** to outputs **Y** using **labeled** examples, then generalize to unseen inputs.



Sub-type	Output	Example tasks	Algorithms
Classification	Discrete class	Spam vs. not-spam, malware family, image label	Logistic regression, decision trees, random forest, SVM, kNN, naive Bayes, gradient boosting (XGBoost, LightGBM), neural nets
Regression	Continuous value	Price, risk score, latency	Linear, polynomial, ridge/lasso, decision-tree and random-forest regressors, SVR, kNN, gradient boosting, neural nets

Worked example — linear regression

The simplest supervised model. Every input (a temperature) is paired with a known output (ice-cream sales), so the model learns the input→output mapping from labeled examples, then predicts on unseen inputs.

```

supervised_regression.py PYTHON

import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Labeled data: each temperature (feature X) is paired with a known sales figure (label y)
temperature = [12, 15, 18, 20, 22, 25, 27, 30, 32, 35]
ice_creams_sold = [18, 24, 31, 38, 45, 57, 66, 78, 88, 105]

X = [[value] for value in temperature] # scikit-learn expects a 2D feature matrix
y = ice_creams_sold

# Hold out part of the data so we can test on examples the model never trained on
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train) # LEARN: find the line (slope + intercept) of best fit
predictions = model.predict(X_test) # APPLY: use the learned line on unseen inputs

print("Predicted ice-cream sales:", predictions.round(2))
print("Actual ice-cream sales: ", y_test)
print(f"Mean squared error: {mean_squared_error(y_test, predictions):.2f}")

plt.scatter(X, y, label="Real data")
plt.plot(X, model.predict(X), color="tomato", label="Model prediction")

```

```
plt.xlabel("Temperature (C)"); plt.ylabel("Ice creams sold")
plt.title("Temperature vs. Ice-Cream Sales")
plt.legend()
plt.show()
```

Runnable with `scikit-learn` and `matplotlib`. The dataset is deliberately tiny for illustration — with 10 points, `test_size=0.2` leaves only 2 test samples, so the error figure is noisy. Real evaluation needs more data or cross-validation.

fit vs. predict — the universal ML rhythm. `fit` is training: it looks at the data and adjusts the model's internal parameters to capture the pattern. `predict` just applies those learned parameters to new input. Think *studying* vs. *taking the test* — and it's always `fit` first (calling `predict` on an unfitted model raises `NotFittedError`).

What `fit` actually stores differs by algorithm, but the shape is identical across all three paradigms below: linear regression learns a **slope + intercept**; k-means learns a set of **cluster centers**; the RL agent learns a **Q-table**. Same rhythm, different learned artifact.

Security Lens

Supervised learning depends entirely on **label integrity**. Corrupt the labels or inject mislabeled samples (**data poisoning**) and the model learns the attacker's intended behavior — e.g., a poisoned training set teaching a malware classifier that a specific attacker tool is "benign."

2.3 · Unsupervised Learning

Find structure in **unlabeled** data — patterns, groupings, or compressed representations discovered on its own.

- **Clustering** — group similar items (k-means, hierarchical, DBSCAN, Gaussian mixture models). Log clustering, user segments.
- **Dimensionality reduction** — compress many features into a few (PCA, t-SNE/UMAP, autoencoders).

- **Association rules** — find items that frequently occur together (Apriori, FP-Growth). Classic market-basket analysis; in security, co-occurring processes, alerts, or commands.
- **Anomaly detection** — flag data points that don't fit learned "normal" (Isolation Forest, Local Outlier Factor, one-class SVM, autoencoders).

Worked example — k-means clustering

The unsupervised counterpart to the regression above. The key difference: there is **no y**. We hand the model only features and ask it to discover groupings on its own — the model invents the labels rather than being given them.

```
unsupervised_kmeans.py PYTHON

import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Customer data: (annual income $1000s, yearly spending score 1-100). No labels – nothing
# tells the model which group anyone belongs to; that is what makes this unsupervised.
income = [15, 16, 17, 55, 58, 60, 62, 85, 88, 90]
spending = [80, 85, 82, 45, 50, 48, 52, 15, 20, 18]

X = [[inc, spend] for inc, spend in zip(income, spending)]

# n_clusters is a hyperparameter you choose – the model won't decide k for you
model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(X) # LEARN: iteratively settle 3 cluster centers

labels = model.labels_ # which discovered group each point landed in
centers = model.cluster_centers_ # the center of each discovered group

print("Cluster assignments:", labels)
print("Cluster centers:", centers.round(1))

plt.scatter(income, spending, c=labels, cmap="viridis", label="Customers")
plt.scatter(centers[:, 0], centers[:, 1], color="red", marker="X", s=200, label="Cluster centers")
plt.xlabel("Annual income ($1000s)"); plt.ylabel("Spending score")
plt.title("Customer Segmentation (K-Means Clustering)")
plt.legend()
plt.show()
```

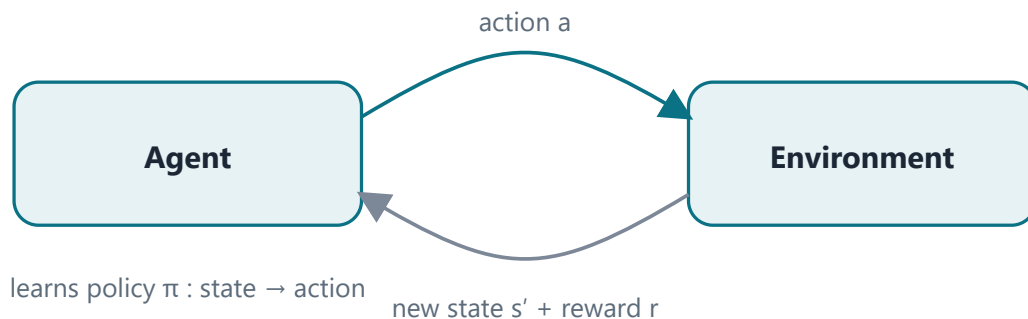
The model separates the three natural customer types (low-income/high-spending, mid/mid, high-income/low-spending) without ever being told they exist. The cluster numbers (0, 1, 2) are arbitrary — interpreting what each group *means* is left to you. Choosing `k` is itself a modeling decision (elbow method, silhouette score); there's no single "correct" answer the way there is with labeled data.

Security Lens — your home turf

Unsupervised **anomaly detection** underpins UEBA, beaconing detection, and endpoint/DNS outlier hunting. Two things to internalize: (1) "normal" is defined by training data, so **baseline poisoning** lets an attacker slowly drift the baseline until malicious activity looks normal (boiling-frog); (2) the real defensive challenge is *precision*, not just recall — low precision buries the SOC.

2.4 · Reinforcement Learning (RL)

An **agent** learns by **interacting** with an **environment**, taking **actions**, and receiving **rewards**. The goal is a **policy** that maximizes cumulative long-term reward — learning by trial and error, not from a fixed labeled set. Formalized as a **Markov Decision Process (MDP)**.



The core loop. It also underpins how LLMs are aligned (RLHF) and how reasoning models are trained (RL on verifiable rewards such as passing tests or correct math answers).

The central tension is **exploration vs. exploitation**: try new actions to find better rewards, or exploit known-good ones. You'll meet RL in game-playing (AlphaGo), robotics, control, **RLHF** for LLMs, and the training of reasoning models.

Algorithms fall into two families. **Value-based** methods learn how good each action is in each state and act greedily on those values (Q-learning, SARSA, Deep Q-Networks). **Policy-based** methods learn the action-selection policy directly (policy gradient, PPO). **Actor-critic** methods combine the two: an actor proposes actions and a critic scores them.

Worked example — Q-learning on a grid world

RL has no dataset and no labels at all. An **agent** starts in the top-left of a 4×4 grid and must reach the goal in the bottom-right, learning purely from **rewards** through trial and error. The "agent" isn't one variable — it's the combination of its memory (the `Q`-table), its decision rule (the *policy*, i.e. the explore/exploit choice), and its learning update. The `step()` function is the *environment*: the agent acts on it but can't change its rules.

```
reinforcement_qlearning.py PYTHON

import numpy as np
import random

# The ENVIRONMENT: a 4x4 grid. Agent starts at cell 0, goal is cell 15.
GRID_SIZE = 4
N_STATES = GRID_SIZE * GRID_SIZE # 16 cells
GOAL = N_STATES - 1 # cell 15
ACTIONS = ["up", "down", "left", "right"]

def step(state, action):
    # Given a cell and a move, return the new cell and the reward.
    row, col = divmod(state, GRID_SIZE)
    if action == "up": row = max(row - 1, 0)
    if action == "down": row = min(row + 1, GRID_SIZE - 1)
    if action == "left": col = max(col - 1, 0)
    if action == "right": col = min(col + 1, GRID_SIZE - 1)
    new_state = row * GRID_SIZE + col
    reward = 10 if new_state == GOAL else -1 # reaching goal is good; every step costs a lit
    return new_state, reward, new_state == GOAL

# The Q-TABLE: the agent's memory. One row per cell, one column per action, all zeros to start
Q = np.zeros((N_STATES, len(ACTIONS)))

alpha = 0.1 # learning rate — how much new experience overrides old
gamma = 0.9 # discount — value of future reward vs. immediate
epsilon = 0.2 # exploration — chance of a random move instead of the best-known one
```

```

# TRAINING: Let the agent play many episodes of trial and error.
for episode in range(500):
    state = 0
    for _ in range(100):          # cap steps so it can't wander forever
        if random.random() < epsilon:    # explore (random) ...
            action_idx = random.randint(0, 3)
        else:                          # ... or exploit (best known move)
            action_idx = int(np.argmax(Q[state]))

        new_state, reward, done = step(state, ACTIONS[action_idx])

        # The Q-Learning update: nudge this value toward reward now + best future value
        best_next = np.max(Q[new_state])
        Q[state, action_idx] += alpha * (reward + gamma * best_next - Q[state, action_idx])

    state = new_state
    if done:
        break

# The Learned POLICY: follow the best action from each cell.
print("Learned best move from each cell:")
for s in range(N_STATES):
    move = "GOAL" if s == GOAL else ACTIONS[int(np.argmax(Q[s]))]
    print(f"Cell {s:2d}: {move}")

```

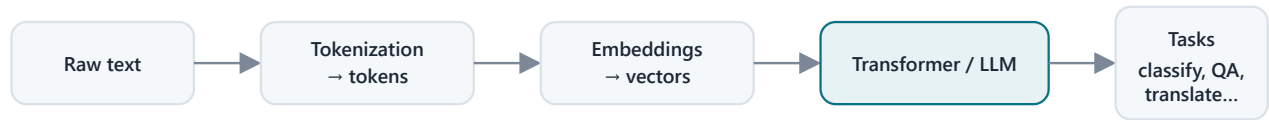
Runnable with `numpy` only. Before training, `Q` is all zeros and the agent wanders randomly; after 500 episodes the Q-values point cleanly toward the goal, so the learned policy is a short, efficient path (down/right) — it discovered the shortest route on its own, because that path collects the fewest -1 penalties before the $+10$ payoff. This is the *tabular* form of RL, which only works when states and actions are few enough to list in a table; real-world RL swaps the table for a neural network, giving **deep RL**, but the act $\rightarrow$ reward $\rightarrow$ update loop is identical.

Security Lens

Reward hacking: an agent exploits a poorly specified reward to score high without the intended behavior. For LLMs, manipulating the feedback/reward signal (or the labelers behind it) is an alignment attack surface. Agentic AI that *takes actions* in real systems can be steered into harmful actions if its objective or observations are tampered with.

2.5 • Natural Language Processing (NLP)

Enabling machines to understand and generate human language.



Evolution: rule-based → statistical (n-grams) → RNN/LSTM → Transformers → LLMs → multimodal and reasoning models. Self-attention weighs every token against every other, which is why it scales.

Security Lens

NLP/LLM systems are where **prompt injection** lives — the top LLM-specific risk. Because these systems don't cleanly separate *instructions* from *data*, attacker-controlled text (a user message, a web page, a retrieved document, an email) can hijack behavior. Also relevant: **sensitive-information disclosure** and **jailbreaks**. Once an LLM is wired to tools, a successful injection stops being a bad answer and becomes an *action* — which is why OWASP moved **Excessive Agency** up to third place in its 2026 LLM list (Appendix B).

2.6 · Computer Vision (CV)

Enabling machines to interpret visual data.

Task	Output
Image classification	One label for the whole image
Object detection	Bounding boxes + labels per object
Semantic segmentation	A class label for every pixel
Instance segmentation	Pixel masks per individual object

Historically powered by **CNNs** (Section 6.3); increasingly by **Vision Transformers (ViT)**.

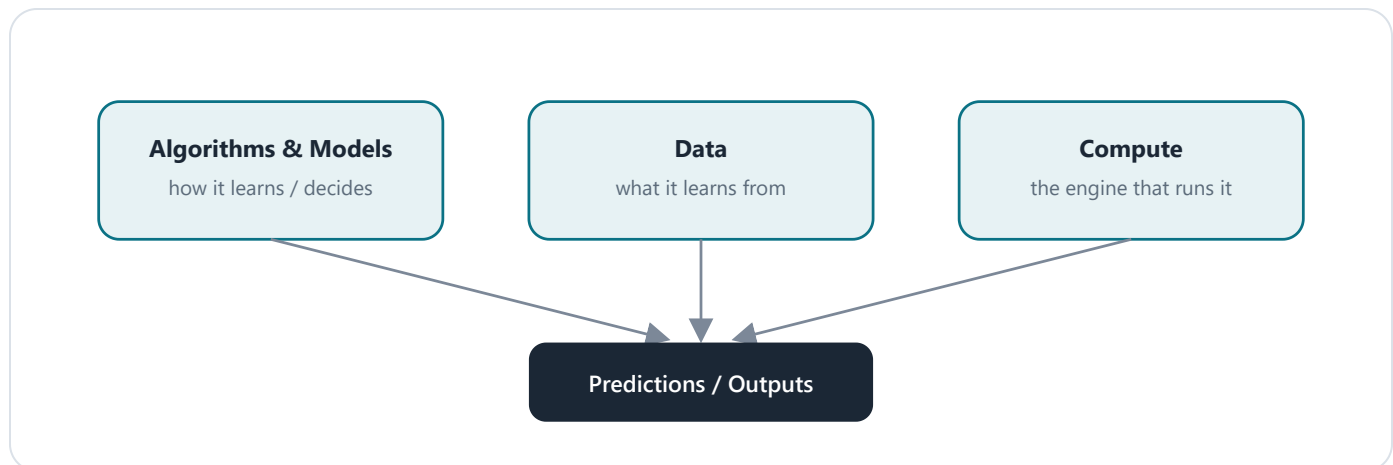
Security Lens

CV is the classic stage for **adversarial examples**: a few crafted pixels (imperceptible to humans) flip a classifier's output. Physical variants — a sticker on a stop sign — matter for camera/vision products. Deepfakes and synthetic media (a generative-CV problem) feed phishing, fraud, and identity attacks.

Section 3

Core Components of AI Systems

Every AI system rests on three interdependent pillars. Weakness in any one degrades the whole — and each is a distinct attack surface.



3.1 · Algorithms and Models

Keep these distinct — people conflate them constantly:

- **Algorithm** = the *procedure/recipe* for learning (gradient descent, decision-tree induction, backpropagation). Fixed and general.
- **Model** = the *learned artifact* produced by running an algorithm on data. A model = **architecture** (structure) + **parameters/weights** (learned numbers).

Algorithm + Data + Compute —train→ Model (architecture + weights)

Security Lens

The trained model is an asset to protect. Threats: **model theft/extraction** (query an API enough to clone it), **model tampering** (alter weights, embed a **backdoor**), and **supply-chain risk** — a pretrained model or checkpoint pulled from a public hub can be maliciously modified/trojaned. Treat model artifacts like signed binaries: provenance, integrity checks, a vetted registry.

3.2 · Data

Data is the fuel, and **data quality dominates model quality** ("garbage in, garbage out").

Dimension	Why it matters
Quantity	Deep models need large datasets to generalize
Quality	Errors, noise, and mislabels propagate into the model
Representativeness	Unrepresentative data → bias and poor real-world performance
Labeling	Supervised learning depends on correct labels
Splits	Train / validation / test must not leak into each other
Drift	Real-world data shifts over time → model decay

Security Lens

Data is the richest attack surface in ML: **training-data poisoning**, **label flipping**, **backdoor triggers** (a hidden pattern forcing a target output), and **privacy attacks** (reconstruct data or infer membership from a trained model). Provenance, integrity, access control, and lineage are AI-security controls — not just governance nice-to-haves.

3.3 · Computing Power

- **Hardware** — GPUs/TPUs provide the parallelism deep learning needs; CPUs orchestrate.
- **Training vs. inference** — training is expensive and periodic; inference is cheaper per call but runs constantly at scale.
- **Distributed training** — large models split across many accelerators.
- **Scaling trend** — capability has grown roughly with more data + more parameters + more compute.

Security Lens

Compute is both a DoS target and a supply-chain concern. LLM apps face **unbounded resource consumption** ("denial of wallet" — driving up inference cost) and model DoS via expensive queries. The training infra, GPU clusters, MLOps tooling, and dependencies form a software supply chain needing the same hygiene as any CI/CD system.

Section 4

Introduction to Machine Learning

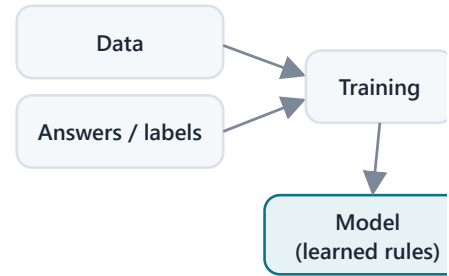
4.1 · What is Machine Learning?

Machine Learning (ML) is the subset of AI where systems **learn patterns from data** to improve at a task, without being explicitly programmed for every rule. A common working definition (paraphrasing Tom Mitchell): *a program learns from experience E on a task T , measured by performance P , if its performance on T improves with E* . The term is usually credited to **Arthur Samuel (1959)**.

TRADITIONAL PROGRAMMING



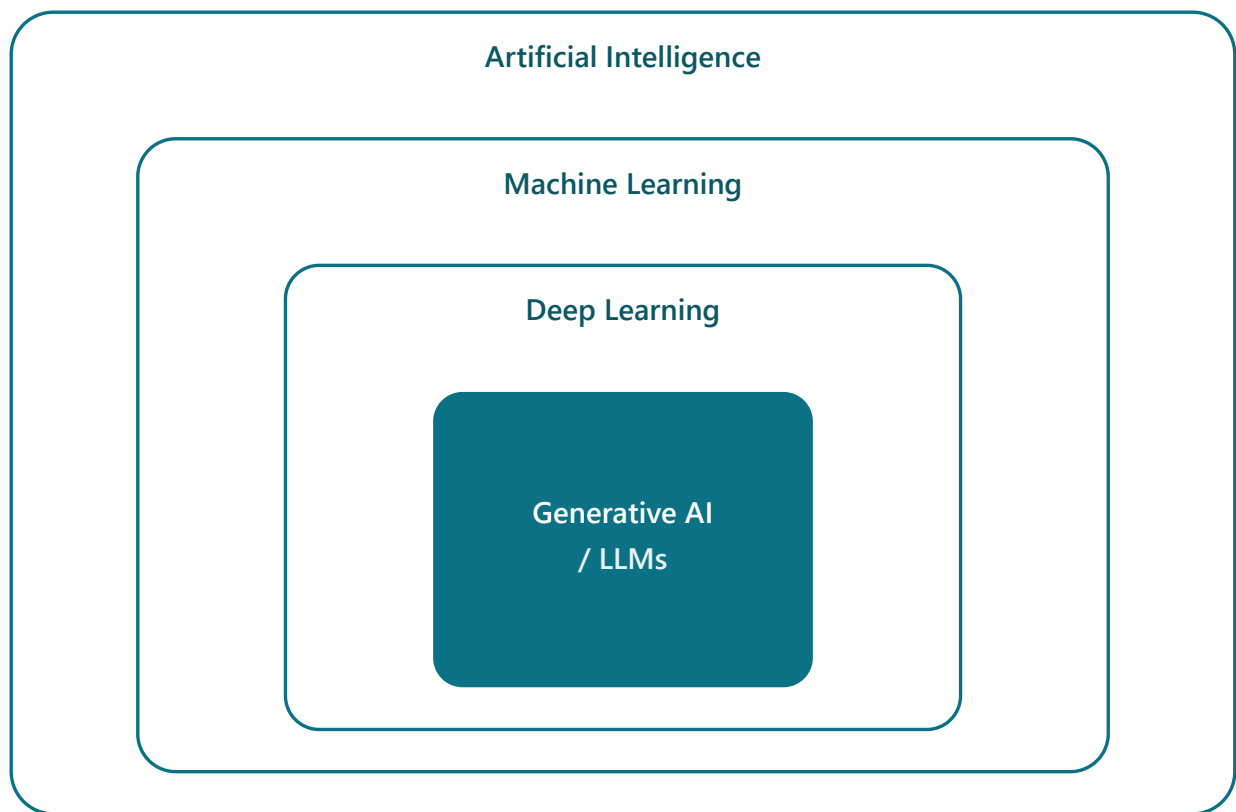
MACHINE LEARNING



Traditional code: humans write the rules. ML: the machine derives rules from data + examples.

4.2 · Differences between AI, ML, and DL

These are **nested subsets**, not competitors.



	AI	ML	Deep Learning
Scope	Broadest — includes rule-based systems	Subset of AI that learns from data	Subset of ML using deep neural nets

	AI	ML	Deep Learning
Feature engineering	N/A or manual	Often manual	Automatic (learned representations)
Data appetite	Varies	Moderate	Large
Interpretability	High (symbolic)	Moderate	Low (opaque)

4.3 · Key ML Concepts

- **Features & labels** — features are input variables; the label is the target you predict.
- **Train / validation / test split** — train to *learn*, validate to *tune*, test to *honestly measure*. Test data stays unseen until the end.
- **Overfitting vs. underfitting** — overfitting memorizes training data (incl. noise) and fails on new data (high variance); underfitting is too simple to capture the pattern (high bias). The **bias–variance tradeoff** balances them.
- **Parameters vs. hyperparameters** — parameters are learned (weights); hyperparameters are chosen by you (learning rate, tree depth, layer count).
- **Loss + gradient descent** — loss quantifies "how wrong"; gradient descent iteratively adjusts parameters to reduce it.
- **Regularization** — penalize complexity to curb overfitting.

Precision vs. recall — essential for security work

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

Precision = $TP / (TP + FP)$ — *of what I flagged, how much was real?* (controls alert fatigue)

Recall = $TP / (TP + FN)$ — *of all real threats, how many did I catch?* (controls misses)

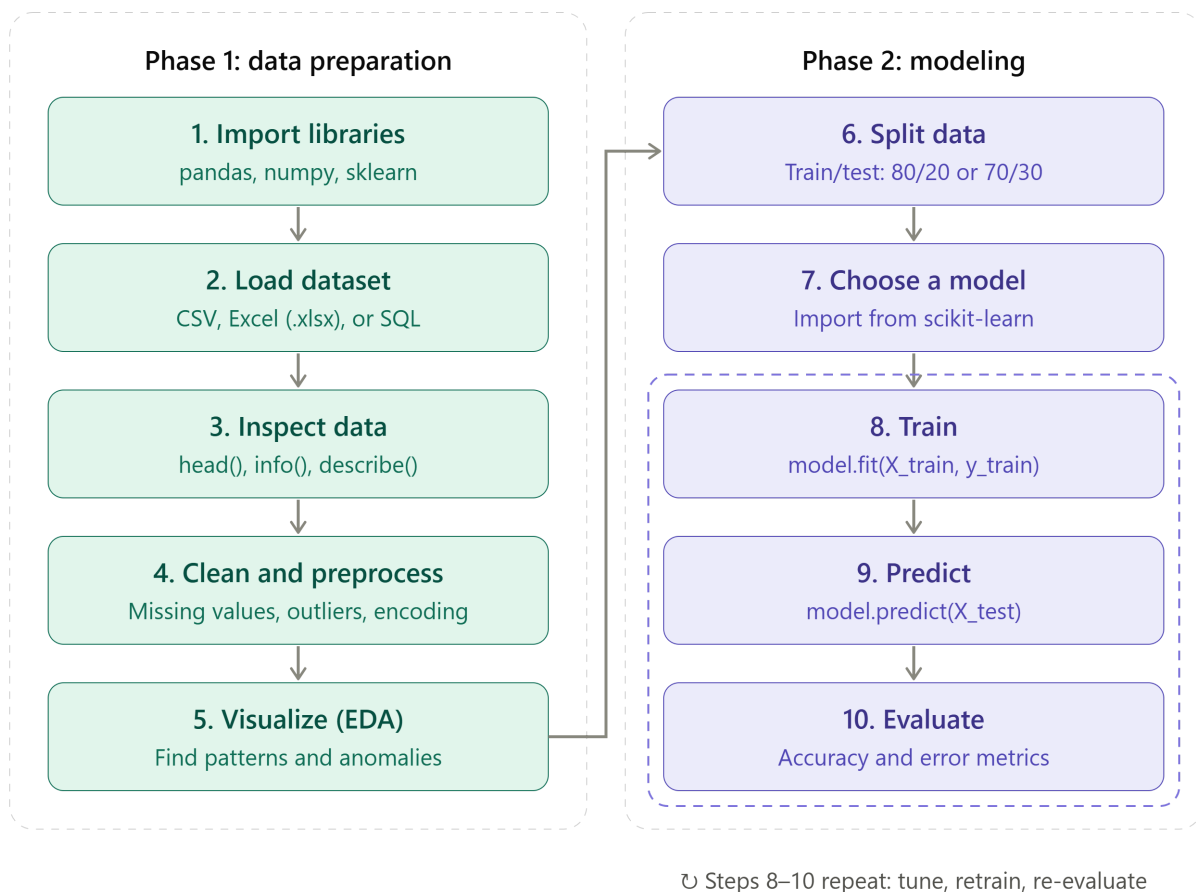
F1 = harmonic mean of precision and recall.

Security Lens

In detection engineering, the precision/recall balance *is* the product. High recall + low precision buries the SOC; high precision + low recall misses intrusions. Overfitting has a security twist too: an overfit model **memorizes training data**, enabling **membership inference** and **data-extraction** — a privacy leak, not just an accuracy problem.

4.4 • The End-to-End ML Workflow

Every concept above shows up in the same ten-step workflow, whatever the algorithm. Phase 1 turns raw data into something a model can learn from; Phase 2 fits, tests, and iterates. The loop at the end is the normal state of the work: you tune hyperparameters, retrain, and re-evaluate until the metrics are good enough, then move on.



Phase 1 prepares the data; Phase 2 builds and evaluates the model. Steps 8–10 repeat while you tune.

Two corrections to the textbook version of this list.

Scale and impute after the split, not before. Many tutorials put standardization and missing-value filling in the EDA step. If you compute a mean, median, or scaler on the full dataset, information from the test set leaks into training and your scores look better than reality. Fit these transformers on the training set only, then apply them to the test set. A scikit-learn `Pipeline` does this for you.

Pick the metric for the problem. Accuracy only suits balanced classification. Regression uses error metrics (MAE, RMSE, R^2). Imbalanced problems — fraud, intrusion, phishing — need precision, recall, F1, or PR-AUC (Section 4.3), because a model that always predicts "benign" can score 99% accuracy.

leak_free_workflow.py

PYTHON

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
```

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Step 6 first: split BEFORE any statistic is computed from the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# Imputer and scaler live inside the pipeline, so they only ever see training data
model = make_pipeline(SimpleImputer(strategy="median"),
                      StandardScaler(),
                      LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)                # step 8: train
y_pred = model.predict(X_test)             # step 9: predict
print(classification_report(y_test, y_pred)) # step 10: precision / recall / F1

```

Assumes `X` (numeric features) and `y` (class labels) are already loaded.

`stratify=y` keeps the class ratio the same in both splits, which matters when the positive class is rare.

Security Lens

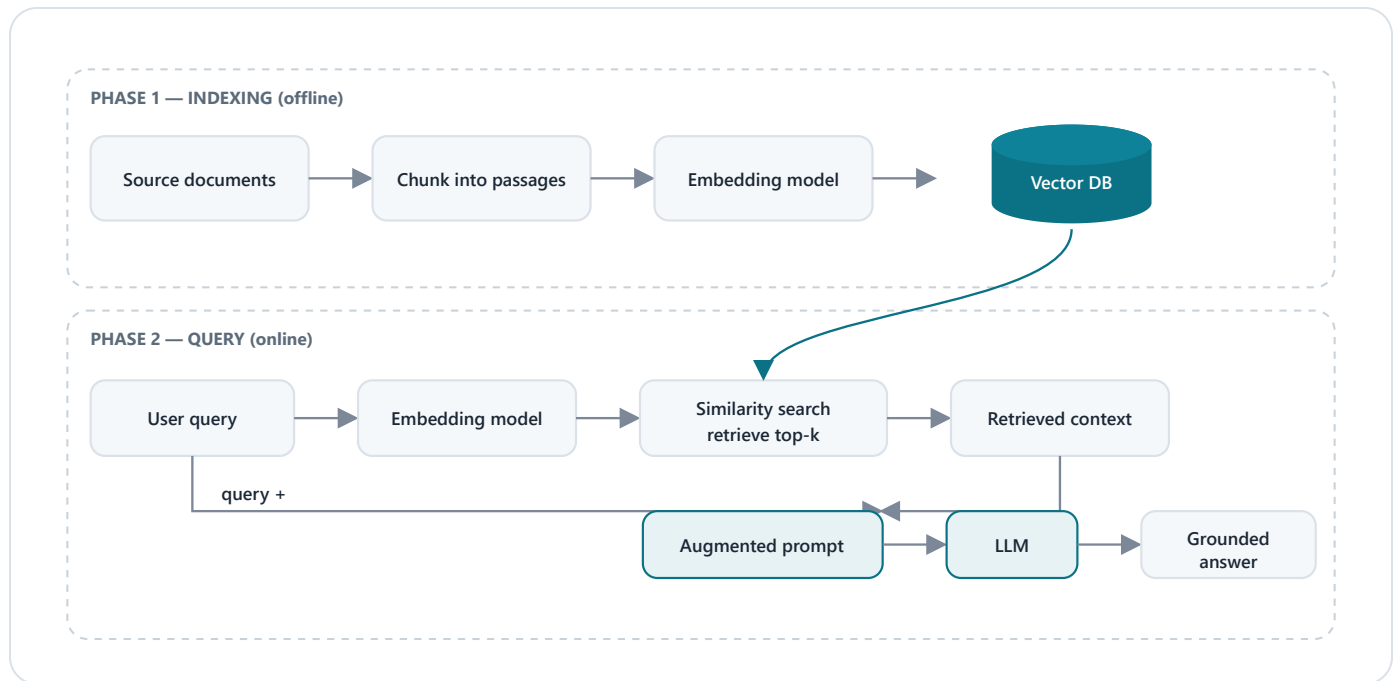
Each phase maps to a control point. **Load** is where poisoned or unvetted data enters, so track provenance and hash datasets. **Clean and preprocess** is where outlier removal can silently delete the rare malicious samples you most need to learn from. **Split** errors (leakage, or near-duplicate samples landing in both train and test) inflate detection metrics and hide real-world failure. **Evaluate** should include adversarial and drift testing, not just a held-out score, and the retraining loop is where baseline poisoning creeps in over time.

Section 5

Retrieval Augmented Generation (RAG)

An LLM's knowledge is **frozen** at training time; on its own it can't access private/current data and may **hallucinate**. **RAG** retrieves relevant external

information at query time and feeds it into the model's context so the answer is **grounded** — without retraining.



Offline indexing builds the vector store; at query time, retrieved chunks are injected into the prompt before generation.

Step by step: ingest & **chunk** documents → **embed** each chunk into a vector (similar meaning → nearby vectors) → **store** in a **vector database** (FAISS, Pinecone, Milvus, Weaviate, Chroma, pgvector) → at query time **retrieve** the top-k most similar chunks → **augment** the prompt → **generate** a grounded answer, ideally with citations.
Benefits: current/private knowledge without retraining, less hallucination, source attribution, cheap updates.

How production RAG usually looks now: hybrid search (keyword/BM25 plus vectors) with a **reranker** to reorder candidates, and increasingly **agentic RAG**, where the model decides when and what to retrieve and can call search or database tools itself, often over the **Model Context Protocol (MCP)**. Long context windows reduce the need for aggressive chunking but haven't replaced retrieval: cost, latency, freshness, and per-user access control still favor it.

Security Lens — RAG concentrates several LLM risks

- **Indirect prompt injection** — retrieved documents become part of the prompt, so an attacker who can plant text in a source (wiki, shared doc, crawled site) can smuggle instructions. Instruction and data share one channel.
- **Knowledge-base / retrieval poisoning** — corrupt the indexed content and you corrupt every downstream answer.
- **Broken access control** — if retrieval doesn't enforce the *user's* permissions, RAG becomes a confused deputy leaking documents. Authorize retrieval per identity, not just at the app boundary.
- **Embedding weaknesses** — embeddings can leak source text (inversion); poorly isolated stores risk cross-tenant leakage.
- **Hidden context exposure** — retrieved chunks, system prompts, and tool definitions sit in the context the user never sees, and attackers try to extract them. OWASP renamed "System Prompt Leakage" to this broader category in 2026.
- **Tool and memory poisoning (agentic RAG)** — when retrieval runs through tools or persistent agent memory, a poisoned tool description or memory entry steers every later step.

Treat retrieved content as **untrusted input**: isolate instructions from data, enforce least privilege on the index, validate and attribute outputs.

Section 6

Basics of Deep Learning

6.1 · What is Deep Learning?

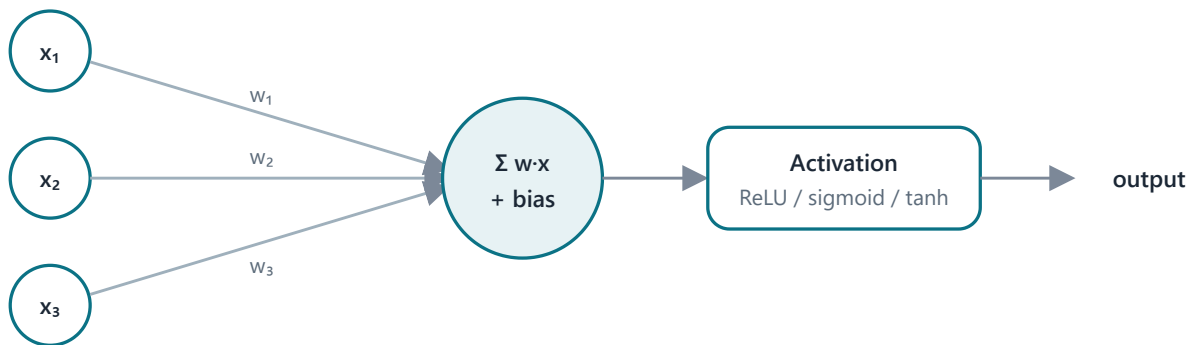
Deep Learning (DL) is the subset of ML that uses **neural networks with many layers**. Its superpower is **representation learning**: instead of humans hand-crafting features, the network **learns hierarchical features automatically** — simple patterns

in early layers combining into complex concepts deeper in. The cost: it's data-hungry, compute-hungry, and opaque.

6.2 · Introduction to Neural Networks

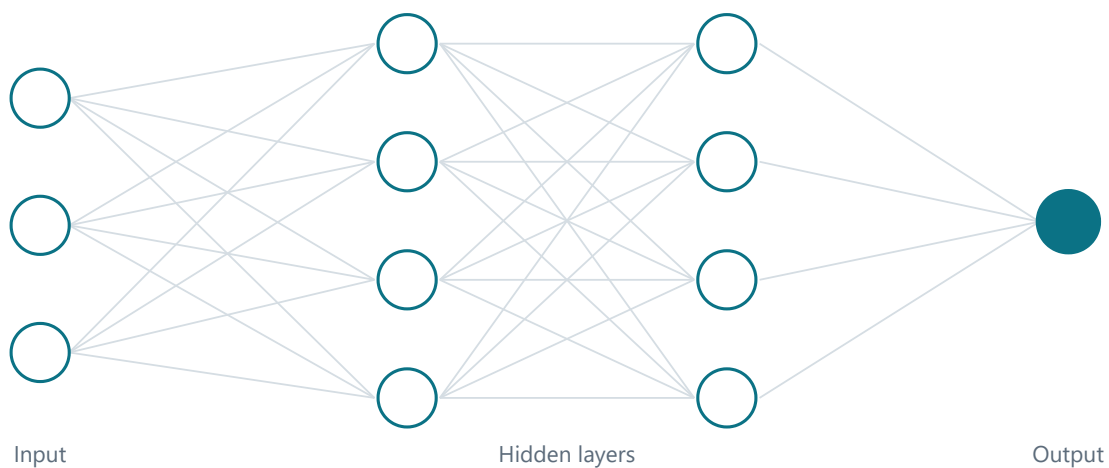
The single neuron

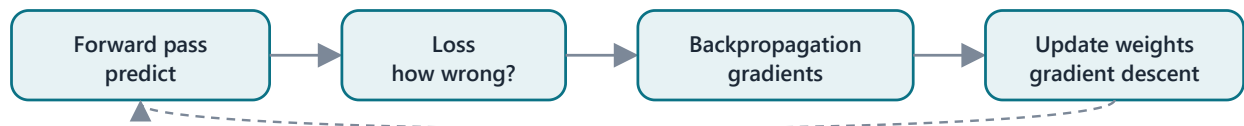
Each neuron multiplies inputs by **weights**, adds a **bias**, then passes the sum through a non-linear **activation function**.



output = activation($\sum(w_i \cdot x_i) + \text{bias}$). Activations add the non-linearity that lets stacked layers learn complex functions.

The network & how it learns





Repeated over many epochs. Common activations: ReLU (hidden layers), sigmoid/softmax (outputs), tanh.

Security Lens

The learned decision boundary is high-dimensional and non-intuitive — which is exactly why **adversarial examples** work. Attackers use the model's own gradients (e.g., FGSM/PGD) to compute tiny perturbations that push an input across the boundary while looking unchanged. White-box (gradient) access makes this easy; black-box attacks work via queries. Defenses (adversarial training, input validation, monitoring) help but none are complete.

6.3 · Convolutional Neural Networks (CNNs)

CNNs are neural networks specialized for **grid-like data**, above all images. They exploit spatial structure instead of wiring every pixel to every neuron.



Three signature operations: **convolution** (small filters/kernels slide over the image producing feature maps — with *parameter sharing* and *translation invariance*), **pooling** (downsamples, keeping strongest signals and adding robustness to shifts), and **fully connected layers** (produce the final classification).



🔒 Security Lens

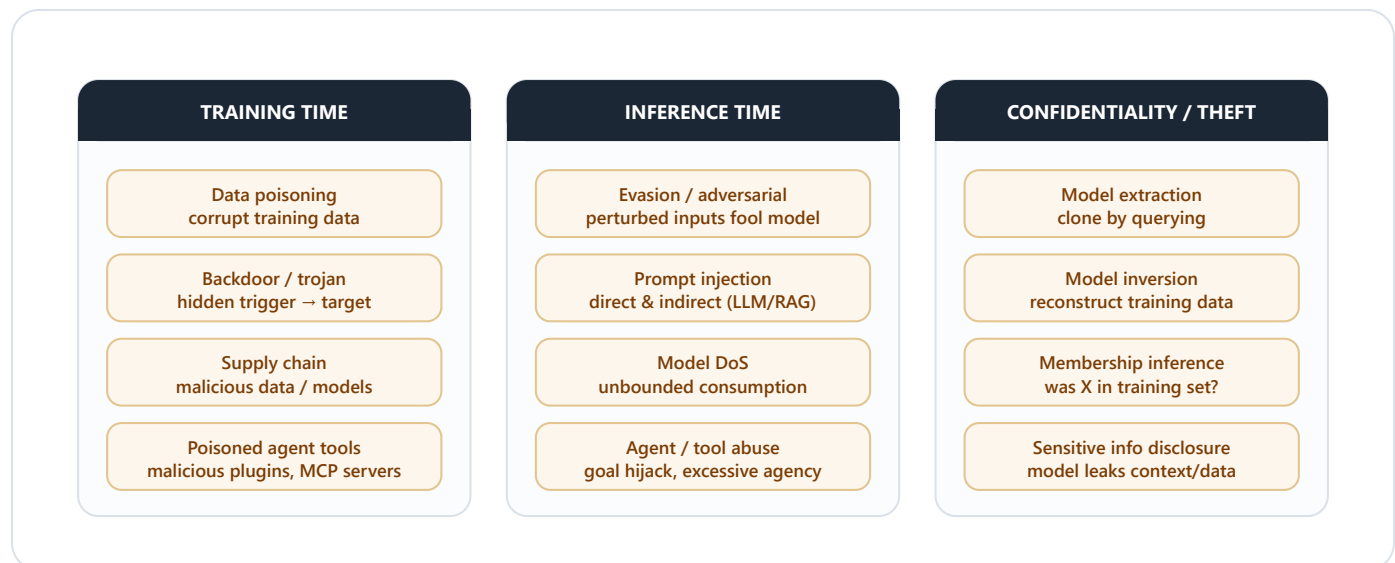
- **Adversarial patches/perturbations** — physical stickers or patterns that fool detectors (e.g., misclassifying a stop sign).
- **Backdoor/trojan attacks** — a CNN trained on poisoned data behaves normally until it sees a hidden trigger, then outputs the attacker's target. Especially dangerous with pretrained models from untrusted sources.
- **Model extraction & inversion** — cloning a deployed vision model or reconstructing training images.

Provenance of both **data and pretrained weights** is the strongest early control.

Appendix A

AI Security Attack Taxonomy

A compact map of the AI-specific threats referenced throughout, organized by *when* in the lifecycle they strike.



Governing Frameworks

Use these as the control catalogs and shared vocabulary of AI security. Status below is as of September 2026. These documents change often, so always cite an item with its edition year (an OWASP "LLM06" means different things in 2025 and 2026).

- **NIST AI Risk Management Framework (AI RMF 1.0)** — organizes AI risk around four functions: **Govern, Map, Measure, Manage**. Best for program-level structure. Pair it with the **Generative AI Profile (NIST AI 600-1, July 2024)**. NIST states that RMF 1.0 is being revised, and in December 2025 it released a preliminary draft **Cyber AI Profile (NIST IR 8596)** that layers AI focus areas (Secure, Defend, Thwart) onto CSF 2.0. Check for the final versions before citing.
- **MITRE ATLAS** (*Adversarial Threat Landscape for AI Systems*) — an ATT&CK-style knowledge base of real-world adversarial-ML tactics and techniques. Best for threat modeling and detection mapping. Its 2025–2026 updates added agent-focused techniques such as publishing poisoned agent tools and harvesting the credentials agents hold.
- **OWASP Top 10 for LLM Applications 2026** (released August 2026, replaces the 2025 list) — LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure, LLM03 Excessive Agency, LLM04 Supply Chain, LLM05 Data and Model Poisoning, LLM06 Unbounded Consumption, LLM07 Misinformation, LLM08 Hidden Context Exposure, LLM09 Vector and Embedding Weaknesses, LLM10 Improper Output Handling. No categories were added or removed, but most moved, so update any control mappings built on 2025 numbering. Best for application-level review.
- **OWASP Top 10 for Agentic Applications 2026** (released December 2025) — risks ASI01–ASI10 for systems that plan, hold memory, and call tools, including agent goal hijack, tool misuse, identity and privilege abuse, memory and context poisoning, insecure inter-agent communication, and rogue agents. It introduces the principle of **least agency**: give an agent only the autonomy its task needs. Use it alongside the LLM list whenever the model can act.
- **ISO/IEC 42001:2023** — the certifiable AI management-system standard (the AI counterpart to ISO 27001). Best when an organization needs auditable AI governance.

- **EU AI Act** (Regulation (EU) 2024/1689) — risk-based law covering anyone placing AI on the EU market. Prohibited practices and general-purpose AI obligations already apply, and Article 50 transparency duties apply from August 2026. The Digital Omnibus (Regulation (EU) 2026/1744, in force since 27 July 2026) pushed high-risk obligations to **2 December 2027** for stand-alone (Annex III) systems and **2 August 2028** for AI embedded in regulated products (Annex I).
-

Appendix C

Python Toolkit by Concept

The concrete Python behind each concept in these notes — the libraries you'd reach for, the key calls, and what they do. Every entry pairs the mainstream *ML tooling* with the *security tooling* a practitioner uses to test or defend it. Snippets are illustrative fragments, not complete programs.

C.1 • Supervised Learning

Section 2.2 showed regression; classification is the other half. `scikit-learn` is the standard entry point, and every estimator shares the same `.fit()` / `.predict()` interface.

supervised_toolkit.py

PYTHON

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)                # Learn the decision boundary
y_pred = clf.predict(X_test)             # predicted class labels
proba = clf.predict_proba(X_test)        # per-class probabilities (for thresholds)

print(confusion_matrix(y_test, y_pred))  # the TP / FP / FN / TN grid
print(classification_report(y_test, y_pred)) # precision, recall, F1 per class
```

What the calls do: `train_test_split` carves out an unseen test set; `cross_val_score` (imported for reference) averages performance over several folds for a more honest estimate on small data; `predict_proba` returns confidence scores you can threshold; `confusion_matrix` and `classification_report` compute the precision/recall/F1 numbers from Section 4.3.

Security tooling — `adversarial-robustness-toolbox` (ART, the most actively maintained option) generates evasion examples to stress-test a classifier, with `foolbox` and `cleverhans` as older research libraries; use held-out label audits and data-versioning (`dvc`) to catch poisoning and label flips.

C.2 · Unsupervised Learning

Same `scikit-learn` foundation, but no `y` — the common shortcut is `.fit_predict()`, which learns and assigns in one call.

```
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest

KMeans(n_clusters=3).fit_predict(X)           # partition into k groups
DBSCAN(eps=0.5, min_samples=5).fit_predict(X) # density clusters; labels noise as -1
PCA(n_components=2).fit_transform(X)          # compress many features down to 2
IsolationForest(contamination=0.01).fit_predict(X) # +1 normal / -1 outlier
```

What the calls do: `KMeans` needs you to pick `k`; `DBSCAN` finds arbitrarily shaped clusters and flags outliers on its own (no `k`); `PCA` reduces dimensionality for visualization or denoising; `IsolationForest` is a go-to unsupervised anomaly detector where `contamination` sets the expected outlier fraction.

Security tooling — `IsolationForest` and `LocalOutlierFactor` are the workhorses behind UEBA and beaconing detection; `river` supports online/streaming models. Watch for *baseline poisoning* — retrain on vetted windows and alert on distribution drift (`evidently`, `alibi-detect`).

C.3 · Reinforcement Learning

The Section 2.4 example was a hand-rolled Q-table. In practice you use a standard environment API (`gymnasium`) and a pre-built agent (`stable-baselines3`), which still expose a `.learn()` / `.predict()` rhythm.

```
rl_toolkit.py PYTHON

import gymnasium as gym
from stable_baselines3 import PPO

env = gym.make("CartPole-v1")           # the ENVIRONMENT (state, action, reward API)
model = PPO("MlpPolicy", env, verbose=0) # the AGENT (a neural-net policy)
model.learn(total_timesteps=10_000)     # train by trial and error (fit-equivalent)

obs, _ = env.reset()
action, _ = model.predict(obs)          # the learned policy chooses an action
```

What the calls do: `gym.make` loads an environment exposing `reset()` and `step(action)`; `PPO` is a modern policy-gradient algorithm (others: `DQN`, `A2C`); `learn` runs the act→reward→update loop for a budget of timesteps; `predict` applies the trained policy. This is *deep* RL — the neural net replaces the Q-table.

Security tooling — RL is the machinery behind *RLHF*, so the attack surface is the reward signal and the labelers behind it. For *agentic* systems that take real actions, sandbox the environment, constrain the action space, and log every action for review — a tampered objective or observation steers behavior.

C.4 • Deep Learning & CNNs

Sections 6.1–6.3. `PyTorch` (shown) and `TensorFlow/Keras` are the two dominant frameworks; you compose layers, define a loss, and step an optimizer.

```
deeplearning_toolkit.py PYTHON

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1), # convolution: Learn image filters
    nn.ReLU(),                                # non-linearity
    nn.MaxPool2d(2),                           # pooling: downsample
    nn.Flatten(),
    nn.Linear(16 * 16 * 16, 10),                # fully connected -> 10 classes
)
loss_fn = nn.CrossEntropyLoss()               # "how wrong" for classification
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) # gradient descent
```

What the calls do: `Conv2d`, `ReLU`, `MaxPool2d` are the three signature CNN operations from Section 6.3; `Linear` is a dense layer producing class scores; `CrossEntropyLoss` is the loss (the "how wrong" from Section 4.3); `Adam` is the optimizer that runs gradient descent over `model.parameters()` (the learned weights).

Security tooling — `ART` and `foolbox` craft adversarial patches and test backdoors. For supply-chain safety, load weights as `safetensors` (not pickle), and scan untrusted checkpoints with `picklescan` or `modelscan` before loading.

C.5 • NLP & LLMs

Section 2.5. The `transformers` library is the hub; tokenizers turn text into IDs, models turn IDs into embeddings or generations.

```
nlp_toolkit.py PYTHON

from transformers import AutoTokenizer, AutoModel

tok = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

tokens = tok("Suspicious login from a new device", return_tensors="pt")
embeddings = model(**tokens).last_hidden_state # contextual vector per token
```

What the calls do: `from_pretrained` downloads a pretrained tokenizer and model; calling the tokenizer converts text into the token-ID tensors the model expects; `return_tensors="pt"` asks for PyTorch tensors; the model call produces embeddings (`last_hidden_state`) you can feed to a classifier or similarity search. For generation you'd use `AutoModelForCausalLM` with `.generate()`.

Security tooling — `garak` and `PyRIT` probe for prompt injection and jailbreaks; `guardrails-ai`, `NeMo-Guardrails`, and Llama Guard validate inputs/outputs. Treat any untrusted text (including retrieved documents) as a potential instruction channel.

C.6 • Retrieval Augmented Generation

Section 5. RAG combines an embedding model with a vector store; retrieved chunks are injected into the prompt before generation.

```
rag_toolkit.py PYTHON
```

```

from sentence_transformers import SentenceTransformer
import chromadb

embedder = SentenceTransformer("all-MiniLM-L6-v2") # text -> vector
col = chromadb.Client().create_collection("docs") # the vector store

col.add(documents=chunks, # PHASE 1: index the chunks
        embeddings=embedder.encode(chunks).tolist(),
        ids=ids)

hits = col.query(query_embeddings=embedder.encode([q]).tolist(), # PHASE 2: retrieve top-k
                 n_results=3)

```

What the calls do: `SentenceTransformer.encode` turns text into embedding vectors

(similar meaning → nearby vectors); `create_collection` makes a vector store; `add`

is the offline indexing phase; `query` is the online retrieval that returns the top-k

most similar chunks to feed the prompt. Alternatives: `faiss`, `pinecone`,

`weaviate`, `pgvector`.

Security tooling — enforce the *user's* permissions at retrieval time (per-identity filters), not just at the app edge, or RAG becomes a confused deputy. Vet indexed content to prevent retrieval poisoning, and keep probing with `garak` / `PyRIT` since retrieved text is the classic *indirect* prompt-injection vector.

Natural next steps: (1) deep-dive adversarial ML with MITRE ATLAS case studies, (2) hands-on prompt-injection and RAG hardening, (3) securing the ML/MLOps pipeline as a software supply chain, (4) agentic AI security — the OWASP Agentic Top 10, MCP tool trust, and least agency.